

Software Engineering Project
Assessment 5

CODING STANDARDS DOCUMENT

Blueberry

AI Dependency Validator

Team Members

Parthiban M — 24BYB0135

Nisha Rathi — 24BYB0137

1. Programming Language and Development Environment

Blueberry is made up of three parts that talk to each other: a backend service, a browser-based dashboard, and a VS Code extension. Each part uses the language that suits it best, but all three follow the same overall coding philosophy: keep code readable, keep modules independent, and keep things simple enough for a two-person team to maintain.

Languages Used

Backend (FastAPI service)	Python 3.11+
VS Code Extension	TypeScript (runs on Node.js)
React Dashboard	TypeScript + React (JSX/TSX)
Database	PostgreSQL 14+ (SQL)
Cache	Redis 6+
Containerization	Docker & Docker Compose

Why These Choices

- Python was chosen for the backend because it has mature libraries for calling the PyPI and GitHub APIs, and FastAPI adds async support so multiple registry checks can run at the same time without slowing things down.
- TypeScript is used for both the extension and the dashboard so the whole front-end side of the project shares one language, one style of writing code, and one set of tools.
- Docker is used so that every team member (and the evaluator) runs the exact same backend environment, regardless of their own machine's operating system.

Development Environment Setup

The table below lists the tools every team member installs before writing code. Keeping this list identical across machines avoids the classic “it works on my computer” problem.

Editor / IDE	Visual Studio Code (v1.85+)
Backend runtime	Python 3.11+, managed with a virtual environment (venv)
Frontend runtime	Node.js (LTS version) with npm
Local services	PostgreSQL and Redis, run via Docker Compose
API testing	VS Code REST Client or Postman, for testing FastAPI endpoints
Version control	Git, with a shared GitHub repository

Everyone runs “docker compose up” to start Postgres and Redis locally, so nobody needs to install and configure a database server by hand. This is the single command that gets a new team member from a fresh clone of the repository to a working local environment.

2. Coding Standards Followed

The goal of these standards is simple: any team member should be able to open a file and understand what it does without needing the author to explain it out loud. Because the backend and frontend use different languages, each has its own style guide, but the underlying principles are the same.

General Principles (apply to all code)

- Use clear, descriptive names for variables, functions, and files — e.g. `check_package_exists()` rather than `chk()` or `doStuff()`.
- Keep functions small and focused on one job. If a function is doing three different things, it should probably be three functions.
- Write a short comment above any piece of logic that isn't immediately obvious — comments should explain “why,” not repeat “what” the code already says.
- Never hardcode secrets (API keys, database passwords, tokens). These live in environment variables, loaded from a local `.env` file that is never committed to Git.
- Handle errors explicitly — for example, if the PyPI API times out, the code should return a clear “could not verify” result instead of crashing the whole request.
- Keep the four backend checks (PyPI, GitHub, similarity, risk score) independent of each other, matching the modular design from the Design Document, so any one of them can be tested or changed without breaking the others.

Python Standards (Backend)

- Follow PEP 8, Python's official style guide, for naming, spacing, and layout.
- Use `snake_case` for variables and functions (`check_pypi_metadata`), and `PascalCase` for classes (`PyPIChecker`, `RiskScoreEngine`), matching the class names already used in the Design Document.
- Add type hints to function signatures so it's clear what goes in and what comes out, e.g. `def get_metadata(name: str) -> PackageMetadata`:
- Format code automatically with Black, and check style issues with a linter such as Flake8, before committing.
- Write a short docstring for every public function and class, describing its purpose in one or two lines.

```
def check_package_exists(package_name: str) -> bool:
    """Return True if the package is published on PyPI, else False."""
    response = pypi_client.get(f"/pypi/{package_name}/json")
    return response.status_code == 200
```

TypeScript Standards (Extension & Dashboard)

- Use camelCase for variables and functions (extractPackageName), and PascalCase for classes, interfaces, and React components (RiskScoreCard).
- Enable TypeScript's strict mode so type mistakes are caught while writing code, not after running it.
- Use ESLint for catching common mistakes and Prettier for automatic formatting, so every file looks consistent no matter who wrote it.
- Keep React components small: one component should render one clear piece of UI (for example, a single FlaggedPackageRow rather than one giant Dashboard component that does everything).

Naming and File Conventions

Item	Convention	Example
Python files	snake_case	pypi_checker.py
Python classes	PascalCase	PyPIChecker
Python functions/variables	snake_case	get_metadata()
TypeScript/React files	PascalCase for components	FlaggedPackageRow.tsx
TypeScript variables/functions	camelCase	sendCheckRequest()
Git branches	kebab-case with a type prefix	feature/pypi-checker
Environment variables	UPPER_SNAKE_CASE	GITHUB_API_TOKEN

3. Project Structure and Module Organization

The repository is split into three top-level folders — one per part of the system — plus a shared folder for project documents. This mirrors the module breakdown from the Design Document, where the VS Code Extension, the FastAPI Backend, the Data Layer, and the Dashboard are treated as separate, independently testable groups.

Repository Layout

```
blueberry/
|-- backend/                                # FastAPI service
|   |-- app/
|   |   |-- checkers/
|   |   |   |-- pypi_checker.py
|   |   |   |-- github_checker.py
|   |   |   |-- similarity_engine.py
|   |   |-- scoring/
|   |   |   |-- risk_score_engine.py
|   |   |-- models/                # PackageMetadata, RepositoryHealth, etc.
|   |   |-- routes/                # /check, /dashboard endpoints
|   |   |-- cache/                 # Redis access
|   |   |-- db/                   # PostgreSQL access
|   |   |-- main.py                # FastAPI app entry point
|   |-- tests/
|   |-- requirements.txt
|
|-- extension/                              # VS Code extension
|   |-- src/
|   |   |-- listeners/             # Editor event listener
|   |   |-- parser/               # AST / import parser
|   |   |-- client/               # Backend API client
|   |   |-- ui/                   # Inline decorations, diagnostics
|   |   |-- extension.ts          # Extension entry point
|   |-- package.json
|
|-- dashboard/                             # React + TypeScript dashboard
|   |-- src/
|   |   |-- components/
|   |   |-- services/             # API calls to backend
|   |   |-- App.tsx
|   |-- package.json
|
|-- docker-compose.yml                 # Spins up Postgres + Redis
|-- docs/                             # SRS, Design Document, this document
```

Backend Module Responsibilities

Each backend module has exactly one job, matching the class diagram in the Design Document. This keeps the codebase easy to extend — adding a new registry later means adding a new checker module, not rewriting existing ones.

Module	Responsibility
pypi_checker.py	Confirms a package exists on PyPI and fetches its metadata
github_checker.py	Checks whether the linked GitHub repository is active and maintained
similarity_engine.py	Runs Levenshtein distance checks against popular package names
risk_score_engine.py	Combines the three check results into one explainable score
db/	Reads and writes package metadata and scoring history in PostgreSQL
cache/	Reads and writes cached registry lookups in Redis

Guiding Rule for Adding New Code

Before adding a new file, ask: “which existing module does this belong to?” If it doesn't fit cleanly into any of them, it is probably a new module and should get its own folder, rather than being bolted onto an unrelated one.

4. Version Control Setup

The project uses Git for version control, hosted on GitHub in a single shared repository. the branching strategy is intentionally lightweight, just enough structure to avoid stepping on each other's work, without the overhead a larger team would need.

Repository & Branches

Hosting	GitHub (private repository, both team members as collaborators)
main	Always stable and working. Only updated by merging a tested feature branch.
dev	Integration branch. Feature branches merge here first for combined testing.
feature/<name>	One branch per feature or module, e.g. feature/pypi-checker, feature/dashboard-ui

Branching Workflow

- Create a new branch off dev for each task: `git checkout -b feature/github-checker`
- Commit small, focused changes as work progresses, rather than one giant commit at the end.
- Open a Pull Request (PR) from the feature branch into dev when the task is complete.
- The other team member reviews the PR before it is merged — even on a two-person team, a second pair of eyes catches mistakes.

Commit Message Convention

Commit messages follow a simple “type: short description” format, so the project history is easy to scan later:

```
feat: add PyPI existence check
fix: handle GitHub API timeout in github_checker
docs: update SRS with new NFR
refactor: split risk score engine into smaller functions
test: add unit tests for similarity engine
```

Other Git Practices

- `.gitignore` excludes environment files (`.env`), virtual environments, `node_modules`, and any build output — so secrets and generated files never get committed.
- No secrets, API keys, or passwords are ever committed to the repository, even in a private repo — they are stored in environment variables instead.
- Tags are used to mark milestones, e.g. `week-3-extension-complete`, so progress lines up with the six-week development timeline.

5. Initial Module Implementation

The very first module to be implemented is the PyPI Checker, since it is the foundation the rest of the backend depends on.

Example: PyPI Checker Module

The example below follows the standards from Section 2: snake_case naming, type hints, a docstring, and explicit error handling instead of letting the request fail silently.

```
# backend/app/checkers/pypi_checker.py
import httpx
from app.models.package_metadata import PackageMetadata

PYPI_BASE_URL = "https://pypi.org/pypi"

async def check_package_exists(package_name: str) -> bool:
    """Return True if the package is published on PyPI."""
    async with httpx.AsyncClient() as client:
        response = await client.get(f"{PYPI_BASE_URL}/{package_name}/json")
    return response.status_code == 200

async def get_package_metadata(package_name: str) -> PackageMetadata | None:
    """Fetch and return metadata for a package, or None if it doesn't exist."""
    async with httpx.AsyncClient() as client:
        response = await client.get(f"{PYPI_BASE_URL}/{package_name}/json")

    if response.status_code != 200:
        return None

    data = response.json()
    return PackageMetadata(
        publish_date=data["releases"][data["info"]["version"]][0]["upload_time"],
        version_history=list(data["releases"].keys()),
        download_count=data.get("info", {}).get("downloads", {}).get("last_month",
0),
        repo_url=data["info"].get("project_urls", {}).get("Source", ""),
    )
```

What This First Module Delivers

- A working FastAPI project skeleton that the rest of the backend builds on.
- A PackageMetadata data model, reused later by the GitHub checker and the risk score engine.
- A pattern (async function, type hints, explicit None-on-failure) that later modules — the GitHub checker and similarity engine — copy for consistency.
- A first unit test (e.g. checking a known real package like “requests” exists, and a made-up name doesn't), setting up the testing habit for the rest of the project.

